
MICROMOUSE ROBOT DESIGN PROJECT

Autonomous Micromouse Robot

From Maze Sensing to Intelligent Navigation

Birzeit University

Department of Electrical & Computer Engineering

SUPERVISOR

Dr. Wasel Ghanem

TEAM MEMBERS

Lujain Alawneh • Safia Salameh • Anees Hammoudeh
Dana Obaid • Mohammad Abutteen



PROJECT GOAL

A Compact Autonomous Micromouse

Build a small robot that solves the instructor's maze safely — without human control and without hitting the walls.

Autonomous

Finds its way through the maze by itself.

Compact

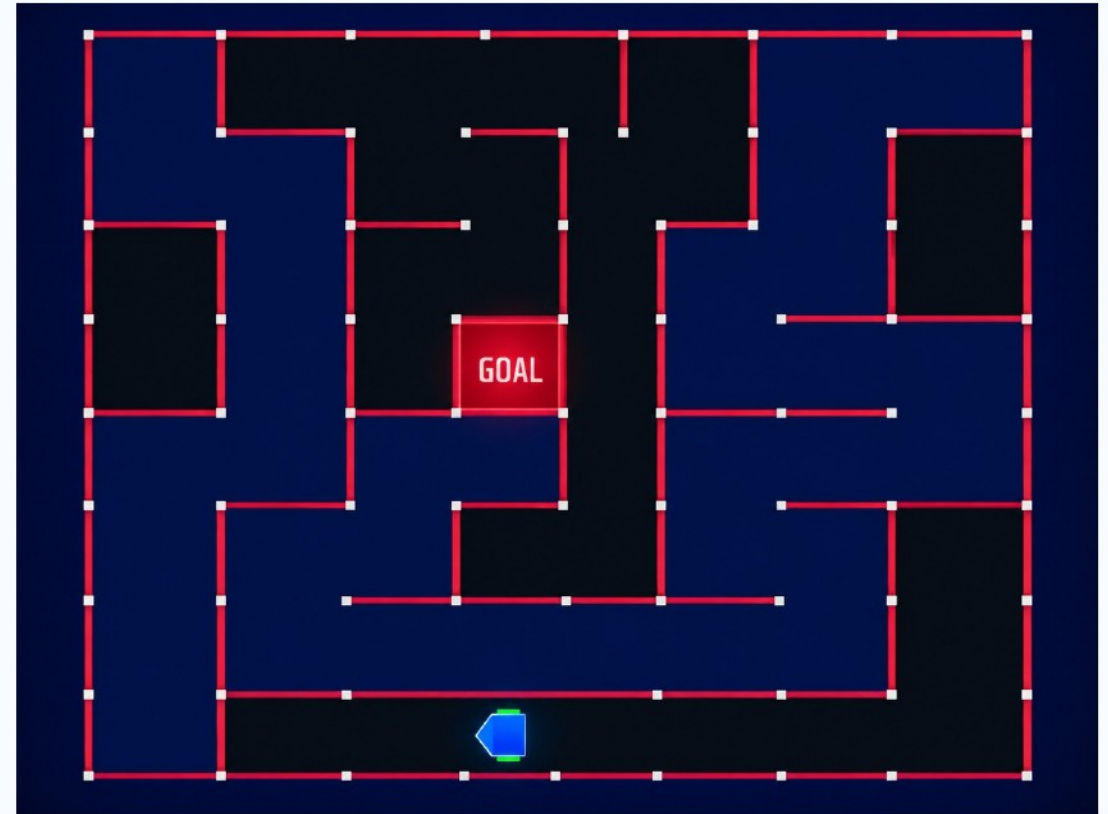
Small size, light weight, and tight component layout.

Safe Motion

Detects walls and avoids collisions.

Goal Driven

Moves toward the target cell efficiently.



COMPONENT 01

ESP32 — Main Controller

The brain that connects sensing, decision-making, and motion control.



ESP32 30-Pin Development Board

Role in our robot

- **Collects sensor feedback**

Reads wall distance, heading, and wheel movement data.

- **Runs the control logic**

Uses feedback to keep motion stable and accurate.

- **Sends motor commands**

Outputs PWM and direction signals to the motor driver.

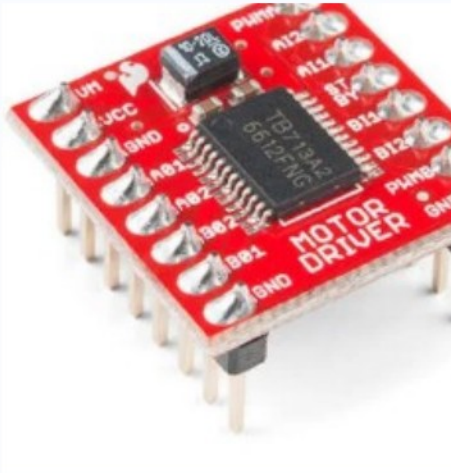
Why ESP32?

Compact • enough GPIO pins • I²C support • PWM outputs • real-time control

COMPONENT 02

TB6612FNG — Motor Driver

The interface that converts ESP32 commands into real motor motion.



TB6612FNG Dual DC Motor Driver

Why do we need it?

- The ESP32 cannot drive the motors directly.
- It provides the higher current the motors need.

How it works

INPUTS

ESP32 control • PWM • direction signals



TB6612FNG

Supplies motor current and switches direction



OUTPUTS

Drives both motors • speed • direction

Its role in our robot

- Controls motor speed using PWM.
- Controls motor direction for forward, backward, left, and right motion.

COMPONENT 03

Voltage Regulator

Mini DC-DC Buck Step-Down Converter Module

5V-30V input → stable 5V output



Actual regulator module used in the robot

What it does

- Steps the battery voltage down to 5V.
- Supplies safe power to the ESP32 and sensors.
- Keeps the control electronics stable while motors are running.

Why we need it

Because the 9V battery is higher than what the 5V electronics need, the regulator provides a stable supply.

COMPONENT 04

N20 DC Motors — Movement System

The actuators that turn driver signals into wheel movement.



N20 Micro DC Gear Motors + Wheels

Why do we need them?

- They move the robot through the maze.
- The gearbox gives high torque in a small body.

How it works

INPUTS

Power from driver • PWM-controlled voltage



DC MOTORS

Convert electrical energy into rotation



OUTPUTS

Left/right wheel motion • speed • direction

Its role in our robot

- Drives the left and right wheels.
- Enables forward motion, turning, and speed control.

COMPONENT 04
COMPONENT 05

VL6180X ToF Sensors

Front and side distance sensing

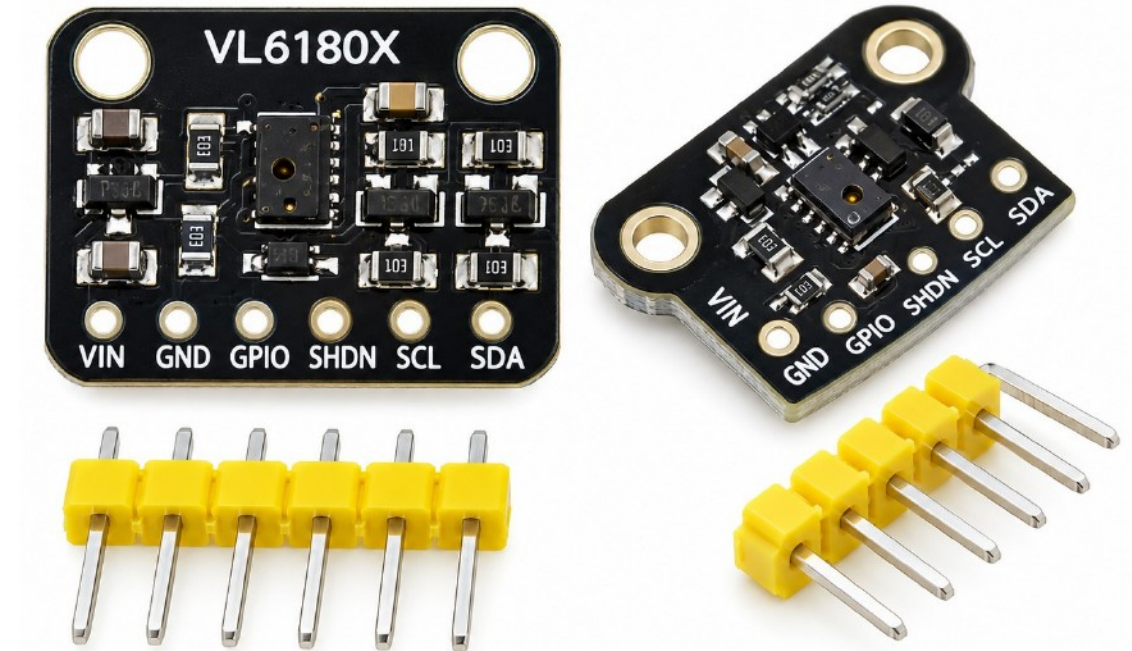
Measures nearby walls so the robot can move safely through the maze.

What it does

- Measures distance to maze walls.
- Front sensor watches ahead.
- Side sensors keep the robot centered.

Why we need it

- Avoids hitting walls.
- Helps choose safe turns.
- Sends readings to ESP32 using I²C.



Front view + angled side view; pin names are readable

Main pins: VIN · GND · GPIO · SHDN · SCL · SDA

BN0055 IMU

Orientation and heading feedback

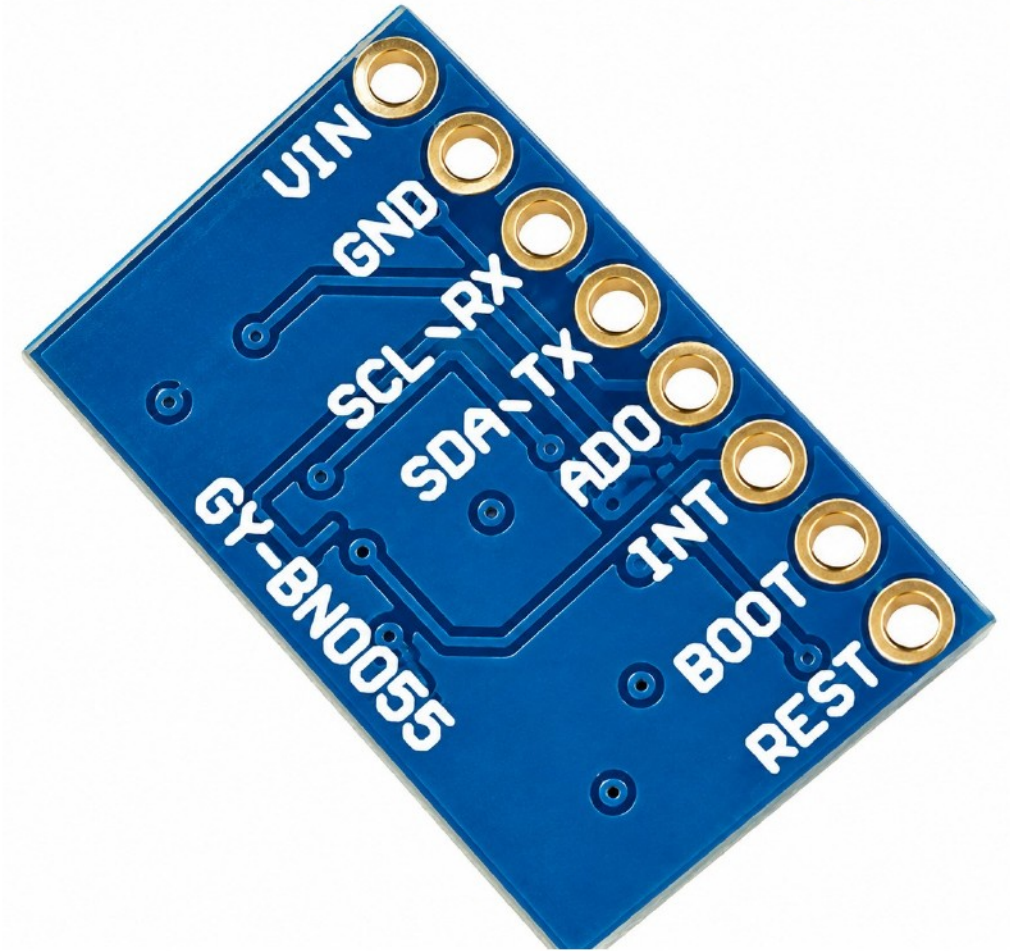
Gives the robot direction and motion feedback for straighter movement and accurate turns.

What it does

- Measures heading and rotation.
- Senses tilt and motion.
- Sends data to ESP32 using I²C.

Why we need it

- Makes 90° turns more accurate.
- Corrects drift while driving forward.
- Keeps the path stable inside the maze.



Pin names shown clearly on the board

**VIN · GND · SCL/RX · SDA/TX · AD0 · INT · BOOT ·
RESET**

COMPONENT 07

Wheel Encoders

Speed and distance feedback

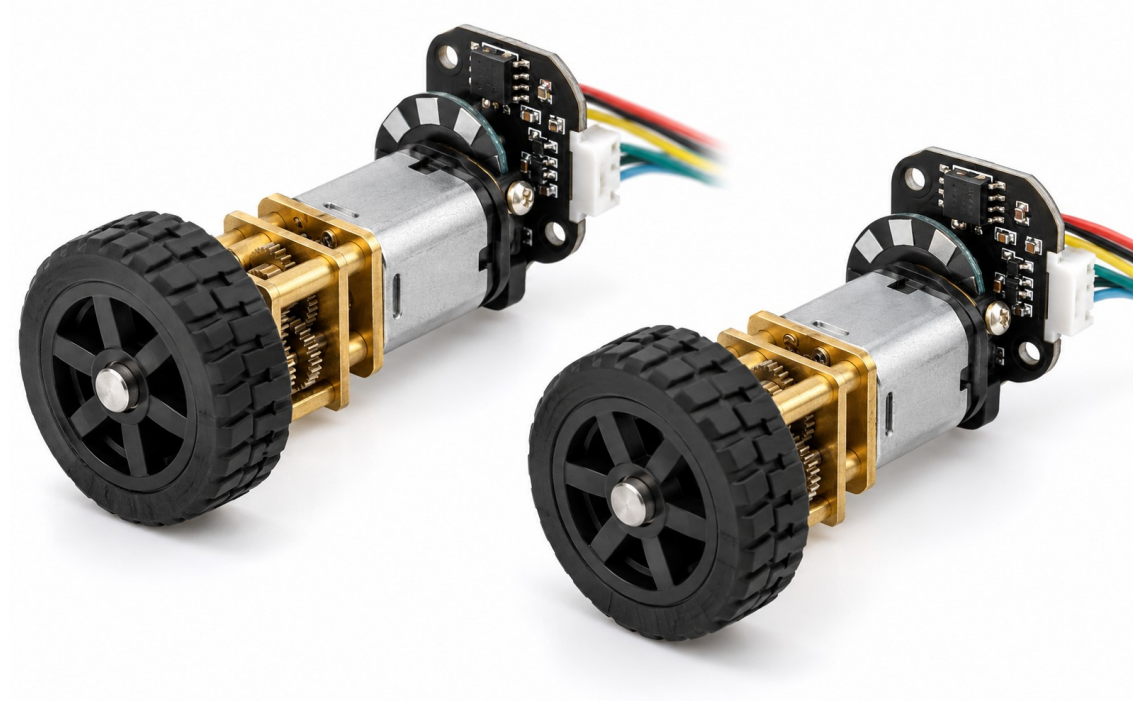
Measures wheel movement so the ESP32 can estimate distance and correct motion.

What it does

- Counts wheel rotations.
- Measures speed and traveled distance.
- Helps both wheels move evenly.

Why we need it

- Keeps straight driving accurate.
- Improves turns and stopping distance.



Hall encoder boards on the back of the N20 motors

Encoder data → wheel speed + distance

COMPONENT 08

9V Rechargeable Li-ion Battery

Power supply for the whole robot

Provides portable power for the controller, sensors, and motor system.

What it does

- Supplies 9V nominal power.
- Rechargeable for repeated maze runs.
- Compact enough to fit inside the chassis.

Why we need it

- Powers the robot without an external supply.
- Feeds the motor driver and regulated electronics.



Main specs: 9V • rechargeable • Li-ion polymer

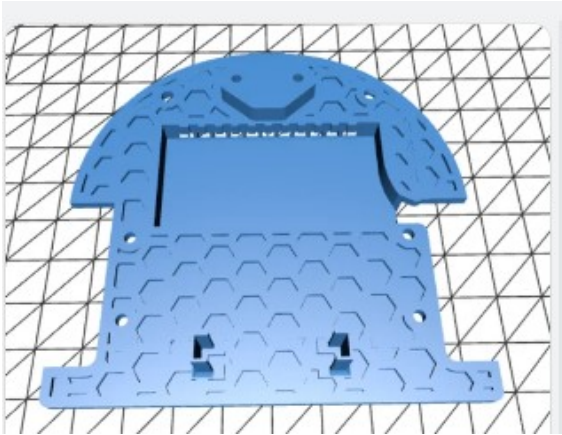
Battery → regulator / driver → ESP32 + sensors + motors

COMPONENT 09

Chassis

Mechanical structure and component mounting

Custom 3D-printed base and top cover hold the robot parts in place.



Chassis base

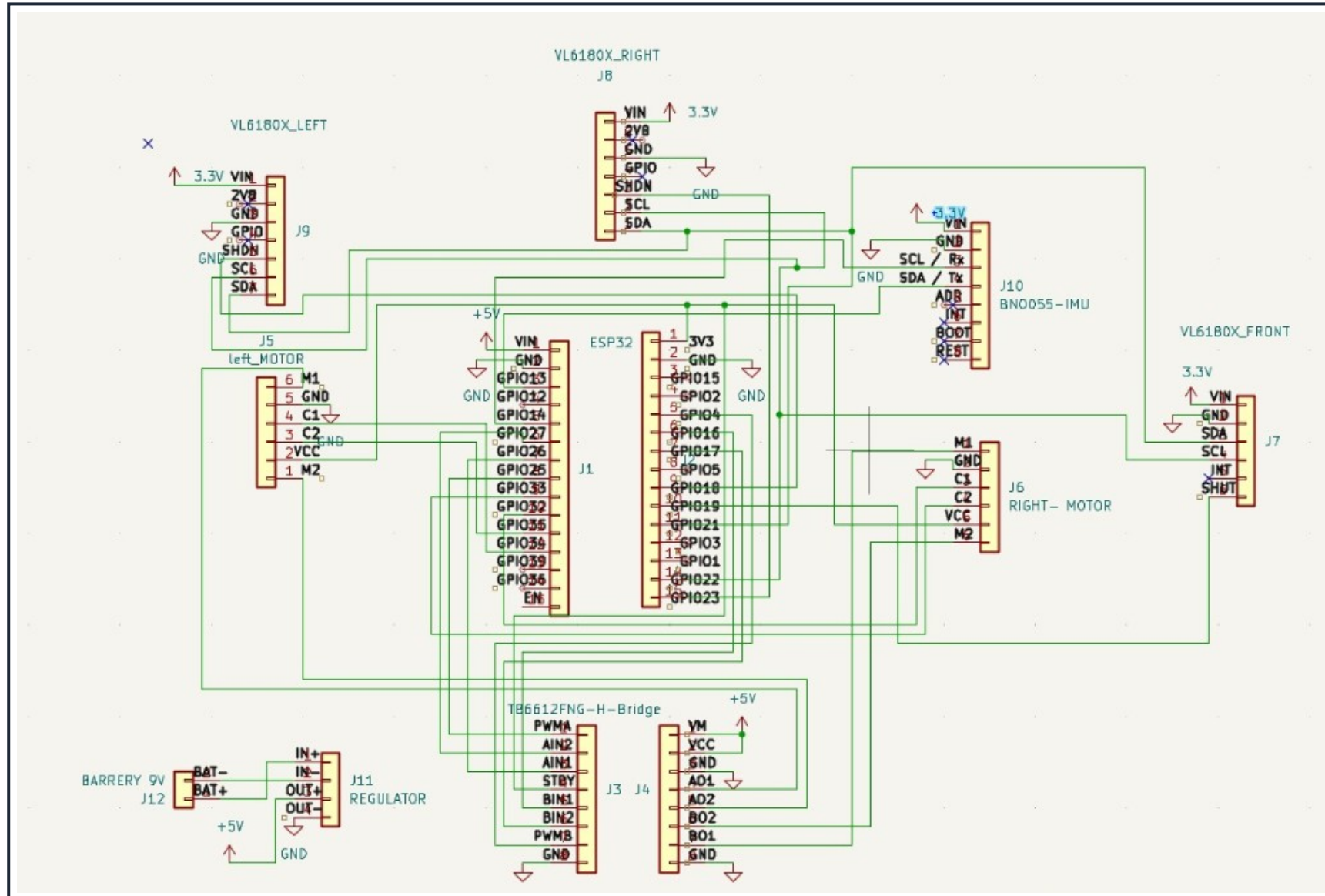


Final top case

- Holds motors, PCB, sensors, and battery.
- Keeps it compact and mechanically stable.

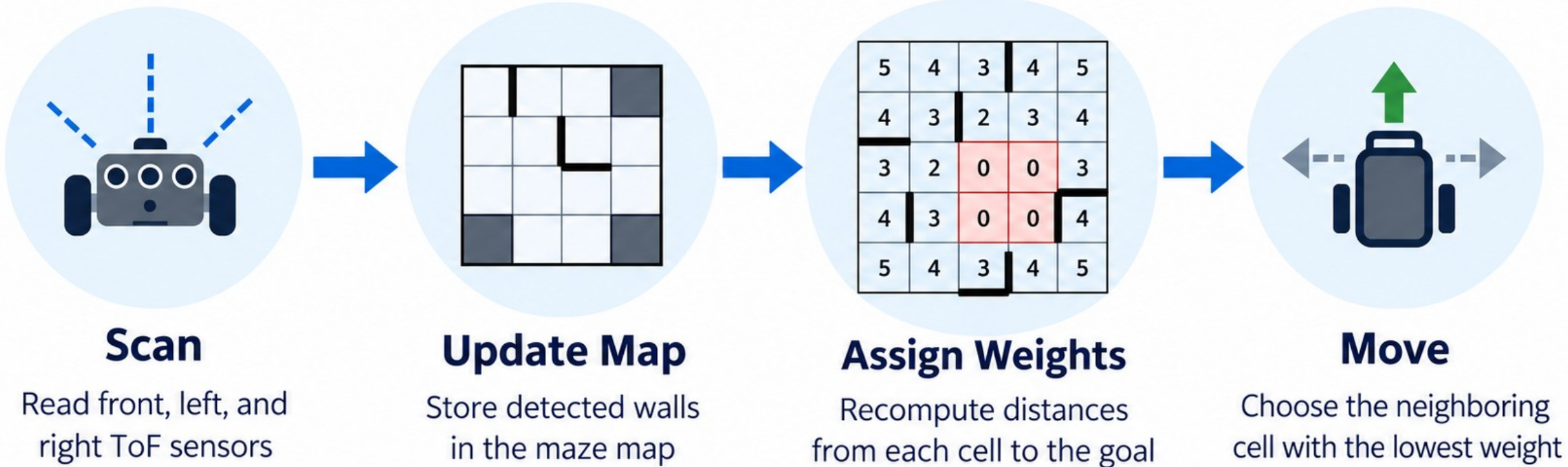
- Provides mounting points and protects internal components.
- Custom design fits our Micromouse layout.

Robot Wiring Diagram — Adopted Connection Layout



Flood Fill Algorithm

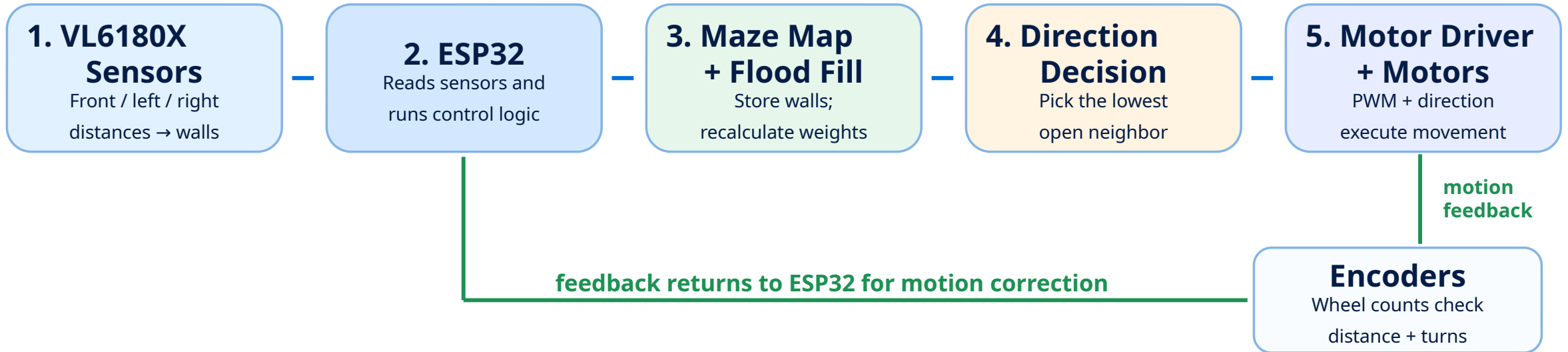
How the robot decides where to move inside the maze



- The map is updated continuously as the robot explores.
- Flood Fill is recalculated whenever new walls are detected.

Flood Fill Integration in Our Robot

How sensing, mapping, planning, and motion control worked together



Project-specific integration

- The robot starts without knowing the full maze; it builds the map cell by cell.
- Whenever new walls are detected, the maze map changes and Flood Fill runs again.
- Flood Fill chooses the next direction, while the ESP32 turns it into motor commands.
- Encoders do not choose the path; they make the chosen movement accurate.

Takeaway: sensors discover walls → ESP32 updates the map → Flood Fill plans → motors move → encoders verify.

How Flood Fill Worked in Our Project

Project-specific navigation cycle, not just the general algorithm

Sensing + Mapping

Planning + Motion

- 1

Read distances
Front, left, and right VL6180X sensors measure wall distance in the current cell.
- 2

Detect walls
The ESP32 converts distance readings into wall / no-wall information.
- 3

Update maze map
Newly detected walls are stored in the robot's internal maze representation.
- 4

Recalculate Flood Fill
If the map changes, cell values are recomputed using the known walls.
- 5

Choose direction
Select the accessible neighboring cell with the lowest Flood Fill value.
- 6

Move accurately
Motors execute the move; encoders help verify one-cell motion and 90° turns.
- updated map

Special notes for our robot

- Unknown maze: the map grows cell by cell.
- New wall → recalculate Flood Fill → re-route.
- Flood Fill chooses direction; encoders make movement and turns accurate.

Power Budget — Battery Requirement

Component	Qty	Operating Voltage	Worst-case Current	Total Worst-case Power
ESP32 30-pin	1	5 V	500 mA	2.50 W
N20 530 RPM Motors	2	5 V	200 mA each	2.00 W
VL6180X ToF Sensors	3	5 V	40 mA each	0.60 W
BMI160 IMU	1	3.3 V	0.99 mA	0.003 W ≈
TB6612FNG Motor Driver	1	5 V (logic)	2.2 mA	0.011 W ≈
MP1584EN Regulator	1	9 V → 5 V	—	Assume ≈90% efficiency
9V Battery (Source)	1	9 V	Must supply ≈ 570–630 mA	—
TOTAL LOAD				≈ 5.11 W

We calculated the power requirements of the main Micromouse components to estimate the total system load and determine the battery requirements.

The estimated total load was approximately **5.11 W**, with the **9V battery required to supply around 570–630 mA**.

This calculation helped us select a suitable power source with an appropriate safety margin.

Cost Calculation

Item No.	Component	Qty	Unit Price (₺)	Total Price (₺)
1	ESP32 30-pin	1	30₺	30₺
2	N20 530 RPM Motors	2	60₺	120₺
3	VL6180X ToF Sensors	3	30₺	90₺
4	BNO055 IMU	1	borrowed	borrowed
5	TB6612FNG Motor Driver	1	15₺	15₺
6	5V Fixed Output Buck Board	1	19₺	19₺
7	9V Battery	1	25₺	25₺
8	wires	1	8₺	8₺
9	Chassis	1	25₺	25₺
TOTAL COST			332₺ < 120\$	

Challenges & Solutions

What we fixed during mechanical assembly, wiring, motor testing, and motion tuning

1

Chassis Reprint

Problem

The first printed chassis had a measurement error. We tried drilling the holes manually, but the part cracked.

Fix

We corrected the dimensions and reprinted the chassis.

2

Sensor Rewiring

Problem

After assembly, the sensors did not operate correctly. During troubleshooting, the wiring became hard to track.

Fix

We disconnected the wiring and rebuilt the sensor connections cleanly.

3

Motor Power Fault

Problem

Motor testing failed because the shared VCC/GND soldering caused an unstable power connection.

Fix

We traced the power path and repaired the common VCC/GND solder joints.

4

90° Turn Tuning

Problem

The robot could not perform consistent 90° turns at first.

Fix

We repeatedly tested and tuned the code and encoder-based motion until turns became reliable.

Takeaway Each issue was isolated, fixed, and retested before moving to the full robot integration.

THE END

